from ayoub boussih

VEILLE JAVASCRIPT

Fonctionsprêtesà l'emploi • DOM • Événements • Méthodes d'array

Guide pratique pour créer une petite bibliothèque de livres et comprendre le code pas à pas.

OBJECTIF Comprendre les méthodes essentielles, savoir quand les utiliser et les réemployer dans un projet web.

PLAN DU GUIDE

PARTIE	CONTENU
01	Les fonctions intégrées et la validation des données
02	Les méthodes de chaînes de caractères (String)
03	Les méthodes de tableaux (Array)
04	Le DOM : sélectionner et modifier la page
05	Les événements et les fonctions de rappel
06	Mini-projet : ajouter, afficher, rechercher et supprimer des livres
07	Erreurs fréquentes et exercices de révision

01 — Fonctions intégrées et validation

JavaScript fournit des fonctions prêtes à l'emploi. Elles évitent de réécrire des opérations courantes.

Fonction	Rôle	Exemple
Number()	Convertit une valeur en nombre.	`Number("42") // 42`
parseInt()	Lit un entier depuis une chaîne.	`parseInt("18px") // 18`
parseFloat()	Lit un nombre décimal.	`parseFloat("3.14") // 3.14`
String()	Convertit une valeur en texte.	`String(25) // "25"`
Boolean()	Convertit une valeur en booléen.	`Boolean(1) // true`
typeof	Indique le type d'une valeur.	`typeof "Salut" // "string"`
Number.isNaN()	Vérifie si une valeur est NaN.	`Number.isNaN(NaN) // true`
Number.isInteger()	Vérifie si la valeur est un entier.	`Number.isInteger(4) // true`
Array.isArray()	Vérifie si une valeur est un tableau.	`Array.isArray([]) // true`

Exemple : récupérer et valider un champ de formulaire

```
constannee = Number(document.querySelector("#annee").value);

if (Number.isInteger(annee) && annee > 0) {
    console.log("Année valide :", annee);
} else {
    console.log("Saisis une année valide.");
}
```

Attention : Number("") renvoie 0. Pour un champ obligatoire, vérifie aussi que le texte n'est pas vide après trim().

Math : quelques outils utiles

Méthode	Résultat / utilisation
Math.round(4.6)	Arrondit au nombre entier le plus proche : 5
Math.floor(4.9)	Arrondit vers le bas : 4
Math.ceil(4.1)	Arrondit vers le haut : 5
Math.max(2, 8, 4)	Renvoie le maximum : 8
Math.min(2, 8, 4)	Renvoie le minimum : 2
Math.random()	Renvoie un nombre supérieur ou égal à 0 et inférieur à 1

02 — Méthodes de chaînes de caractères (String)

Une chaîne de caractères est un texte, par exemple un titre de livre ou le nom d'un auteur.

Méthode	À quoi sert-elle ?	Exemple
trim()	Supprime les espaces au début et à la fin.	" Ada ".trim() → "Ada"

Méthode	À quoi sert-elle ?	Exemple
toLowerCase()	Met le texte en minuscules.	"BONJOUR" → "bonjour"
toUpperCase()	Met le texte en majuscules.	"livre" → "LIVRE"
includes()	Vérifie si un texte contient un fragment.	"JavaScript".includes("Script") → true
startsWith()	Vérifie le début d'un texte.	"Bonjour".startsWith("Bon") → true
endsWith()	Vérifie la fin d'un texte.	"photo.png".endsWith(".png") → true
replace()	Remplace la première occurrence trouvée.	"a-a".replace("a", "b") → "b-a"
replaceAll()	Remplace toutes les occurrences.	"a-a".replaceAll("a", "b") → "b-b"
slice()	Extrait une partie du texte.	"Bonjour".slice(0, 3) → "Bon"
split()	Découpe une chaîne en tableau.	"a,b,c".split(",") → ["a", "b", "c"]

```
const titre = document.querySelector("#titre").value.trim();

if (titre.length === 0) {
    console.log("Le titre est obligatoire.");
}
```

À retenir : les méthodes de String renvoient généralement une nouvelle chaîne ; elles ne modifient pas le texte d'origine.

03 — Méthodes de tableaux (Array)

Un tableau stocke plusieurs valeurs. Dans le projet Bibliothèque, il peut contenir des objets représentant les livres.

Méthode / propriété	Rôle	Exemple
push(x)	Ajoute à la fin.	livres.push(nouveauLivre)
pop()	Retire le dernier élément.	livres.pop()
unshift(x)	Ajoute au début.	nombres.unshift(0)
shift()	Retire le premier élément.	nombres.shift()
splice(i, n)	Retire ou remplace des éléments à partir de l'index i.	livres.splice(1, 1)
slice(d, f)	Copie une portion sans modifier l'original.	nombres.slice(0, 2)
concat()	Fusionne des tableaux.	a.concat(b)
forEach()	Exécute une action pour chaque élément.	livres.forEach(livre => console.log(livre.titre))
map()	Crée un nouveau tableau transformé.	livres.map(livre => livre.titre)
filter()	Crée un tableau des éléments qui passent un test.	livres.filter(livre => livre.annee >= 2020)
find()	Renvoie le premier élément correspondant.	livres.find(livre => livre.id === 2)
findIndex()	Renvoie l'index du premier élément correspondant.	livres.findIndex(livre => livre.id === 2)

Méthode / propriété	Rôle	Exemple
some()	Vrai si au moins un élément passe le test.	nombres.some(n => n < 0)
every()	Vrai si tous les éléments passent le test.	nombres.every(n => n > 0)
reduce()	Réduit le tableau à une valeur (somme, total…).	nombres.reduce((s, n) => s + n, 0)
includes()	Vérifie si une valeur est présente.	[1, 2].includes(2)
indexOf()	Renvoie l'index d'une valeur, ou -1.	["a", "b"].indexOf("b")
join()	Assemble les éléments en une chaîne.	["a", "b"].join(", ")
sort()	Trie le tableau en place.	nombres.sort((a, b) => a - b)
reverse()	Inverse l'ordre en place.	nombres.reverse()
length	Donne le nombre d'éléments.	livres.length

Les quatre méthodes à bien distinguer

Méthode	Question à se poser	Retour
forEach()	Quelle action faire pour chaque élément ?	Aucun nouveau tableau
map()	Quelle nouvelle valeur créer pour chaque élément ?	Nouveau tableau de même longueur
filter()	Quels éléments garder ?	Nouveau tableau, parfois plus court
find()	Quel est le premier élément qui correspond ?	Un élément ou undefined

```
const livres = [

   { titre: "HTML facile", annee: 2021 },
   { titre: "JavaScript moderne", annee: 2024 },
   { titre: "CSS pratique", annee: 2019 }
 ];

 const titres = livres.map(livre => livre.titre);
 const recents = livres.filter(livre => livre.annee >= 2022);
 const js = livres.find(livre => livre.titre.includes("JavaScript"));
 const total = livres.reduce((compteur, livre) => compteur + 1, 0);
```

Attention : sort() et reverse() modifient le tableau original. Pour trier des nombres, utilise (a, b) => a - b ; sans fonction de comparaison, le tri peut être alphabétique.

04 — Fonctions et fonctions fléchées

Une fonction regroupe des instructions que l'on peut réutiliser. Les paramètres sont les entrées ; return renvoie le résultat.

	Exemple
Déclaration	function saluer(nom) { return `Bonjour ${nom}`; }
Expression	const doubler = function(n) { return n * 2; };
Fonction fléchée	const doubler = n => n * 2;
Plusieurs paramètres	const addition = (a, b) => a + b;
Valeur par défaut	const saluer = (nom = "ami") => `Bonjour ${nom}`;

Callback : une fonction passée en argument à une autre fonction.

```
constnombres = [1,  2, 3];

const doubles = nombres.map(nombre => {
    return nombre * 2;
});

// Résultat : [2, 4, 6]
```

- **Paramètre :** le nom de la variable dans la définition, par exemple nom.
- **Argument :** la valeur envoyée lors de l'appel, par exemple saluer("Lina").
- **return :** renvoie une valeur et termine l'exécution de la fonction.
- **Sans accolades :** une fonction fléchée peutretourner directementuneexpression : n=>n * 2.

05 — DOM : sélectionner et modifier la page

Le DOM représente le HTML sous forme d'objets JavaScript. On s'en sert pour lire, créer ou modifier des éléments.

Méthode / propriété	Utilité	Exemple
getElementById()	Sélectionne un élément par son id.	document.getElementById("titre")
querySelector()	Sélectionne le premier élément correspondant au sélecteur CSS.	document.querySelector(".carte")
querySelectorAll()	Sélectionne tous les éléments correspondants.	document.querySelectorAll(".carte")
getElementsByClassName()	Sélectionne par nom de classe.	document.getElementsByClassName("carte")
getElementsByTagName()	Sélectionne par nom de balise.	document.getElementsByTagName("p")
textContent	Lit ou modifie le texte.	titre.textContent = "Mes livres"
innerHTML	Lit ou modifie le HTML intérieur.	zone.innerHTML = "**OK**"
value	Lit la valeur d'un champ de formulaire.	input.value
createElement()	Crée un élément HTML.	document.createElement("li")
append() / appendChild()	Ajoute un élément à la fin.	liste.append(li)

Méthode / propriété	Utilité	Exemple
prepend()	Ajoute au début.	liste.prepend(li)
remove()	Supprime l'élément sélectionné.	carte.remove()
replaceChildren()	Remplace tous les enfants.	liste.replaceChildren()

Classes, styles et attributs

Outil	Exemple	Utilité
classList.add()	carte.classList.add("active")	Ajoute une classe CSS.
classList.remove()	carte.classList.remove("active")	Retire une classe CSS.
classList.toggle()	menu.classList.toggle("ouvert")	Ajoute ou retire selon l'état.
classList.contains()	carte.classList.contains("active")	Vérifie si la classe existe.
setAttribute()	img.setAttribute("alt", "Couverture")	Définit un attribut.
getAttribute()	lien.getAttribute("href")	Lit un attribut.
removeAttribute()	input.removeAttribute("disabled")	Retire un attribut.
style	titre.style.color = "blue"	Modifie un style en ligne.

Naviguer dans le DOM

• element.parentElement : récupère l'élément parent.

• element.children : récupère les éléments enfants.

• element.closest(".carte") : remonte jusqu'au premier ancêtre qui correspond au sélecteur.

• element.nextElementSibling / previousElementSibling : élément frère suivant ou précédent.

Sécurité : pour afficher du texte saisi par l'utilisateur, préfère textContent à innerHTML. N'insère pas directement du contenu non fiable dans innerHTML.

06 — Événements et formulaires

Un événement signale une action : clic, saisie, envoi de formulaire, touche clavier, etc.

Événement	Quand l'utiliser ?	Exemple
click	L'utilisateur clique.	bouton.addEventListener("click", ajouter)
input	La valeur change pendant la saisie.	recherche.addEventListener("input", filtrer)
change	La valeur est validée ou modifiée.	select.addEventListener("change", trier)
submit	Un formulaire est envoyé.	form.addEventListener("submit", ajouter)
keydown / keyup	Une touche est pressée / relâchée.	document.addEventListener("keydown", gererTouche)
focus / blur	Un champ reçoit / perd le focus.	champ.addEventListener("focus", activer)
DOMContentLoaded	Le HTML est analysé et le DOM est prêt.	document.addEventListener("DOMContentLoaded", init)

Outil	Explication
event	Objet qui contient les informations sur l'événement.
event.target	Élément à l'origine de l'événement.
event.currentTarget	Élément sur lequel le gestionnaire a été installé.
event.preventDefault()	Empêche le comportement par défaut, par exemple l'envoi classique d'un formulaire.
event.stopPropagation()	Empêche l'événement de remonter dans la chaîne de propagation.
removeEventListener()	Retire un gestionnaire, à condition de réutiliser la même fonction.

```
const form = document.querySelector("#formLivre");

form.addEventListener("submit", event => {
    event.preventDefault();

    // Lire les champs, valider puis ajouter le livre.
});
```

Pour un formulaire, pense généralement à submit + preventDefault(). Cela évite le rechargement de la page pendant que ton JavaScript traite les données.

07 — Mini-projet : petite bibliothèque

Exemple pédagogique : ajouter des livres, les afficher, rechercher un titre et supprimer un livre.

Structure HTML minimale

```
<form id="formLivre">

   <input id="titre" placeholder="Titre du livre" required>
   <input id="auteur" placeholder="Auteur" required>
   <input id="annee" type="number" placeholder="Année" required>
   <button type="submit">Ajouter</button>

</form>

<input id="recherche" placeholder="Rechercher un livre">
<ul id="listeLivres"></ul>
```

JavaScript

```
let livres = [];
let prochainId = 1;

const form = document.querySelector("#formLivre");
const titreInput = document.querySelector("#titre");
const auteurInput = document.querySelector("#auteur");
const anneeInput = document.querySelector("#annee");
const rechercheInput = document.querySelector("#recherche");
const liste = document.querySelector("#listeLivres");

function renderList(tableau = livres) {
    liste.replaceChildren();

    tableau.forEach(livre => {
        const li = document.createElement("li");
        const texte = document.createElement("span");
        const bouton = document.createElement("button");

        texte.textContent =
            `${livre.titre} — ${livre.auteur} (${livre.annee})`;
        bouton.textContent = "Supprimer";

        bouton.addEventListener("click", () => {
            livres = livres.filter(item => item.id !== livre.id);
            renderList();
        });

        li.append(texte, bouton);
        liste.append(li);
    });
}

form.addEventListener("submit", event => {
    event.preventDefault();

    const titre = titreInput.value.trim();
    const auteur = auteurInput.value.trim();
    const annee = Number(anneeInput.value);

    if (!titre || !auteur || !Number.isInteger(annee) || annee <= 0) {
        alert("Vérifie les informations saisies.");
        return;
    }

    livres.push({
        id: prochainId++,
        titre,
        auteur,
        annee
    });

    form.reset();
renderList(); });


rechercheInput.addEventListener("input", () => {
    const terme = rechercheInput.value.trim().toLowerCase();

    constresultats = livres.filter(livre =>
        livre.titre.toLowerCase().includes(terme) ||
        livre.auteur.toLowerCase().includes(terme)
    );

    renderList(resultats);
});
```

Comment le code fonctionne

Étape	Explication
1. Sélection	querySelector récupère les champs et la liste HTML.
2. Écoute	addEventListener attend l'envoi du formulaire ou la saisie dans la recherche.
3. Validation	trim() retire les espaces inutiles ; Number() convertit l'année.
4. Ajout	push() ajoute un objet livre au tableau.
5. Affichage	renderList() crée les éléments HTML à partir du tableau.
6. Recherche	filter() garde les livres dont le titre ou l'auteur contient le terme recherché.
7. Suppression	filter() conserve tous les livres dont l'id est différent de celui à supprimer.

08 — Sauvegarder avec localStorage

Le tableau en mémoire disparaît au rechargement. localStorage permet de conserver des données simples dans le navigateur.

```
function sauvegarderLivres() {
    localStorage.setItem("mesLivres", JSON.stringify(livres));
}

function chargerLivres() {
    const donnees = localStorage.getItem("mesLivres");
    livres = donnees ? JSON.parse(donnees) : [];
    renderList();
}
```

• JSON.stringify(objet) transforme une donnée JavaScript en texte JSON.

• JSON.parse(texte) reconvertit un texte JSON en donnée JavaScript.

• localStorage.setItem(cle, valeur) enregistre une chaîne.

• localStorage.getItem(cle) lit la valeur enregistrée ou renvoie null si elle n'existe pas.

• Appelle sauvegarderLivres() après un ajout ou une suppression, puis chargerLivres() au démarrage.

Note : localStorage est adapté à un petit exercice local, pas au stockage de mots de passe ou de données sensibles.

09 — Erreurs fréquentes à éviter

Erreur	Pourquoi ?	Bonne pratique
Oublier # ou .	Les sélecteurs CSS distinguent id et classe.	querySelector("#titre") pour un id ; querySelector(".carte") pour une classe.
Oublier preventDefault()	Le formulaire peut recharger la page.	L'appeler dans le gestionnaire submit.
Confondre map et forEach	map crée un tableau ; forEach sert surtout à effectuer une action.	Choisir la méthode selon le résultat voulu.
Utiliser == sans raison	La conversion automatique peut produire des surprises.	Préférer === dans la plupart des comparaisons.

Erreur	Pourquoi ?	Bonne pratique
Utiliser innerHTML avec une saisie	Du contenu inattendu peut être interprété comme du HTML.	Utiliser textContent pour du texte simple.
Oublier trim()	Un champ peut contenir uniquement des espaces.	Nettoyer les champs texte avant validation.
Oublier de redessiner la liste	Les données changent, mais pas l'interface.	Appeler renderList() après chaque modification.

10 — Exercices de révision

Essaie de répondre sans regarder les exemples précédents, puis teste tes réponses dans la console du navigateur. 1.

Quelle différence y a-t-il entre map() et filter() ?

2. Comment sélectionner le premier élément qui possède la classe carte ?

3. Quelle méthode retire les espaces autour d'un titre saisi ?

4. Comment empêcher un formulaire de recharger la page ?

5. Quelle méthode ajoute un livre à la fin du tableau ?

6. Comment trouver le premier livre dont le titre contient « CSS » ?

7. Pourquoi textContent est-il souvent préférable à innerHTML pour une saisie utilisateur ?

8. Comment compter le nombre de livres dans le tableau ?

9. Comment supprimer le livre qui possède un id précis sans utiliser splice() ?

10. À quoi servent JSON.stringify() et JSON.parse() ?

Corrigé rapide

N°	Réponse attendue
1	map transforme chaque élément ; filter conserve ceux qui passent une condition.
2	document.querySelector(".carte")
3	trim()
4	event.preventDefault() dans le gestionnaire submit.
5	tableau.push(element)
6	livres.find(livre => livre.titre.includes("CSS"))
7	textContent affiche du texte sans interpréter la saisie comme du HTML.
8	livres.length
9	livres = livres.filter(livre => livre.id !== idASupprimer)
10	stringify convertit en texte JSON ; parse reconvertit en donnée JavaScript.

Méthode de travail conseillée : lis une section, recopie un exemple, modifie-le, puis explique à voix haute ce qu'il fait. La pratique régulière vaut mieux que mémoriser toutes les méthodes d'un coup.